A Window Is Just a Customer of Your Model

Compact GUI over a tested model (S04)

Computer Science II • Dr. Jeremy P. Evert

October 19, 2026

Today's destination

TODAY'S DESTINATION

- A small model class that passes a `unittest` *without* opening a window.
- A Tkinter view: one label, one button, one callback.
- The trace: click → callback → model method → new state → label updated.
- Break the boundary on purpose, then refactor it back.

▶ *Cue: Say the goal out loud: we are not building a desktop app, we are keeping the boundary clean.*

Core concept: model, view, callback

CORE CONCEPT

Model	Holds the state and the rules. Knows nothing about Tkinter.
View	Shows the state. Has no rules of its own.
Callback	The only bridge: receives one event, calls *one* model method, then refreshes the view.

- *Cue: Ask: where does "you cannot remove an item that is not there" belong? (Model.)*

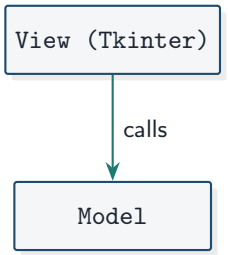
Concept + diagram: who knows about whom

CORE CONCEPT + DIAGRAM

The view *calls* the model.

The model never imports Tkinter and never calls the view.

That one-way arrow is the whole design.



Model first: ten lines, no window

CODE EXAMPLE

```
class Inventory:
    def __init__(self):
        self.items = []

    def restock(self, name):
        self.items.append(name)
        return len(self.items)

    def count(self):
        return len(self.items)
```

- *Cue: Swap in your own world: roster, case file, crew list, shop stock.*

The test passes with no window

CODE EXAMPLE

```
import unittest
from inventory import Inventory

class InventoryTest(unittest.TestCase):
    def test_restock_adds_one(self):
        inv = Inventory()
        self.assertEqual(inv.restock("rope"), 1)
        self.assertEqual(inv.count(), 1)

if __name__ == "__main__":
    unittest.main()
```

Predict before running: the view

PREDICT BEFORE RUNNING

```
import tkinter as tk
from inventory import Inventory
model, root = Inventory(), tk.Tk()
text = tk.StringVar(value="Items: 0")
tk.Label(root, textvariable=text).pack()
def on_restock():
    n = model.restock("rope")    # route to the model
    text.set(f"Items: {n}")      # then refresh the view
tk.Button(root, text="Restock", command=on_restock).pack()
root.mainloop()
```

Predict: three clicks on Restock. What does the label say?

Trace the event

CODE EXAMPLE

```
def on_restock():
    print("click -> callback")
    n = model.restock("rope")
    print(f"model.restock -> {n} items")
    text.set(f"Items: {n}")
    print("label updated")
```

One click: click → callback → model.restock → 1 item → label updated ▶ *Cue:*

Students paste their own trace into the World Bible as evidence.

Common failure: “the GUI is the program”

COMMON FAILURE

Symptom: the only way to know if it works is to run the window and click.

Fix: write the model test first. If you cannot test a rule without a window, the rule is in the wrong place.

Break the boundary on purpose

COMPARE TWO APPROACHES

Boundary broken

```
items = []    # view owns data
def on_restock():
    items.append("rope")
    n = len(items)
    text.set(f"Items: {n}")
```

Boundary kept

```
model = Inventory()
def on_restock():
    n = model.restock("rope")
    text.set(f"Items: {n}")
```

- *Cue: Ask: how would you test the left version without a window?*

Think • Pair • Discuss

THINK • PAIR • DISCUSS

Pick the single most useful action in *your* world: add a colonist, close a case, restock an item. Name the model method, then the one label and one button that would call it.

STOP – STUDENTS WORK NOW

5 minutes. Write the method name and the label text on paper. No Tkinter yet.

Peer swap: plug in your partner's model

DESIGN DECISION

Trade model files with a partner who used the *same method names*. Run your view on their model.

- It works: the contract held (your Week 7 contract test, again).
- It fails: the method names or return values differ. Decide which side should change.

World Bible entry

WORLD BIBLE ENTRY

Rule: The view never holds the rules

Assumption: Every change to world state goes through one model method.
Invariant: The model test passes with no window; the label always shows the model's value.
Revision: Date, and what you moved out of a callback into the model.

Odyssey checkpoint note

ODYSSEY CHECKPOINT

This week's checkpoint: render real model state in a modest view and route one meaningful event through a callback to a model operation. Keep the boundary clear and show the model works on its own.

Evidence: a model class with a window-free `unittest`; your one-label, one-button view; the click trace in your World Bible.

Larger checkpoint, not a desktop-app engineering unit. See the Week 10 gate in Canvas for the rubric; this slide is the pointer, not the rubric.

Recap

RECAP

- Model first, and prove it with a test that needs no window.
- The callback routes one event to one model method, then refreshes.
- Rules in the view are a bug you cannot test.
- Same method names make models swappable (Week 7 contract).

Exit question

EXIT QUESTION

Name one rule in your world that must live in the model, and say how you would test it without opening a window.

Next move: bring a working model and its test to Wednesday.